

第 11 章：交互受限时怎样学习：Model-based、Offline RL、Imitation 与 Preference Alignment

11.1 本章符号说明

符号/缩写	English	中文	本章中的具体含义
RL	Reinforcement Learning	强化学习	通过数据或环境交互学习长期决策策略。
MDP	Markov Decision Process	马尔可夫决策过程	由状态、动作、转移、奖励和折扣构成的决策模型。
MBRL	Model-based Reinforcement Learning	基于模型的强化学习	学习或使用环境模型，再进行规划或生成模拟数据。
MPC	Model Predictive Control	模型预测控制	每个真实时刻只规划有限步，执行第一步后重新规划。
$P(s' s,a)$	Transition Probability	状态转移概率	真实环境从 (s,a) 到下一状态 s' 的概率。
$\hat{P}_\varphi(s' s,a)$	Learned Transition Model	学得转移模型	参数 φ 对真实转移的近似。
$\hat{R}_\psi(s,a)$	Learned Reward Model	学得的环境奖励模型	参数 ψ 对环境 Reward 的近似；不特指 RLHF 的偏好模型。
H	Planning Horizon	规划时域	在模型中向未来模拟或搜索的步数。
D	Fixed Dataset	固定数据集	Offline RL 或 Imitation Learning 可访问的历史样本集合。
$\mu(a s)$	Behavior Policy	行为策略	生成固定数据集的旧策略、人类或多策略混合。
OOD	Out-of-Distribution	分布外	数据覆盖不足的状态、动作或状态动作对。
BC	Behavior Cloning	行为克隆	把专家动作当监督标签做最大似然学习。
DAgger	Dataset Aggregation	数据集聚合	让专家继续标注 Learner 实际访问到的状态。
IRL	Inverse Reinforcement Learning	逆强化学习	从专家行为反推 Reward，再据此求策略。
BCQ	Batch-Constrained deep Q-learning	批约束深度 Q 学习	将 Policy Improvement 限制在数据支持动作附近。
CQL	Conservative Q-Learning	保守 Q 学习	对数据外高 Q 动作施加保守惩罚。
IQL	Implicit Q-Learning	隐式 Q 学习	不显式生成数据外动作，用 Expectile Value 和加权 BC 学策略。
DT	Decision Transformer	决策 Transformer	把轨迹写成条件序列，根据目标回报预测动作。

符号/缩写	English	中文	本章中的具体含义
RTG	Return-to-Go	剩余回报	从当前位置向后的累计回报，DT 的条件输入。
d / done	Terminal Indicator	真终止指示量	$d=1$ 时 Bellman Target 不再 Bootstrap。
$Q_w(s,a), V_v(s)$	Action / State Value Estimate	动作 / 状态价值估计	参数 w, v 的 Critic 估计。
τ_e	Expectile Level	期望分位水平	IQL 中控制 V 向数据动作高 Q 区域偏移的系数。
β_I	IQL Advantage Temperature	IQL 优势温度	控制高优势数据动作的模仿权重。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好定义训练信号并优化语言模型策略。
SFT	Supervised Fine-Tuning	监督微调	先用高质量示范回答训练基础策略。
RM	Reward Model	奖励模型	从偏好对中学习回答质量标量分数的模型。
PPO	Proximal Policy Optimization	近端策略优化	RLHF 在线采样阶段常用的策略优化算法。
SAC	Soft Actor-Critic	软 Actor-Critic	普通在线 Off-policy 算法，本章用它说明为何 Replay 算法不能直接变成 Offline RL。
DPO	Direct Preference Optimization	直接偏好优化	将 KL 正则化偏好目标化为监督式成对分类损失。
KL	Kullback-Leibler Divergence	KL 散度	衡量当前策略与参考策略的分布差异。
x, y^+, y^-	Prompt, Preferred, Rejected Response	提示、偏好回答、被拒回答	一条成对偏好样本。
π_θ, π_{ref}	Current / Reference Policy	当前 / 参考策略	正在优化的策略与固定参考策略。
$r_\omega(x, y)$	Preference Reward Score	偏好奖励分数	参数 ω 的 RM 对回答给出的标量。
$\sigma(z)$	Logistic Sigmoid	Logistic Sigmoid 函数	将分数差变成偏好概率。
β_D	DPO KL Scale	DPO 的 KL 尺度	控制策略相对参考模型变化的强度。

11.2 本章目标

本章不是罗列四组热点名词，而是回答交互受限时的四种问题：

1. 有数据但真实交互贵，能否学一个环境模型反复规划？
2. 完全不能继续交互，只能用固定日志，怎样避免策略利用数据外估值？
3. 没有明确 Reward，但有专家示范，怎样学习策略？
4. 语言任务的好坏来自人类偏好时，RLHF 和 DPO 分别优化什么？

11.3 先分清四个主题改变了哪条假设

主题	能否继续交互	主要监督信号	核心困难
Model-based RL	通常可以，但代价高	Transition / Reward 数据	模型误差被规划放大
Offline RL	不可以	固定 Transition + Reward	OOD 动作与 Bootstrap 外推
Imitation Learning	视方法而定	专家动作或轨迹	Learner 状态分布偏移
Preference Alignment	DPO 不在线交互；PPO-RLHF 会由当前策略采样回答	人类偏好对	偏好噪声、Reward Hacking、策略漂移

它们可以组合。例如先用历史机器人数据训练 Offline Dynamics Model，再做 Model-based Offline RL；也可以先 SFT，再做 PPO-RLHF。但概念边界仍要分清。

11.4 Model-based RL：把真实交互转化为模型内计算

11.4.1 Model-free 和 Model-based 的差别

Model-free RL（无模型强化学习）不显式预测下一状态，直接学习 Value 或 Policy。Model-based RL 显式使用：

$$\hat{P}_{\phi}(s'|s,a), \hat{R}_{\psi}(s,a)$$

再在模型中评估动作序列。优势不是“模型一定更准”，而是同一批真实数据可以支持大量便宜的虚拟 Rollout 或 Planning。

11.4.2 两条典型路线：Planning 与 Dyna

路线一：直接 Planning。给定当前状态，在模型中搜索动作序列：

$$(a_0^*, \dots, a_{H-1}^*) = \arg \max_{a_{0:H-1}} \mathbb{E}_{\hat{P}} [\sum_{h=0}^{H-1} \gamma^h \hat{R}(s_h, a_h)]$$

MPC 每次只执行 a_0^* ，观察真实下一状态后再规划。这会不断用真实观测校正起点，通常比一次性执行整条长计划更稳健。

路线二：Dyna-style Learning（Dyna 式学习）。真实 Transition 既训练 Dynamics，也训练 Value/Policy；随后从已有状态出发，让模型生成额外 Transition，再继续训练 Model-free Agent。

真实交互 → 更新环境模型 → 更新 Agent
→ 模型模拟数据 → 再更新 Agent

11.4.3 为什么 Model Bias 会随 Horizon 累积

假设一维系统真实动态是：

$$s_{t+1} = s_t + a_t$$

学得模型每步都额外高估 0.1：

$$\hat{s}_{t+1} = \hat{s}_t + a_t + 0.1$$

若连续规划 10 步，即使动作完全相同，预测状态也会偏离真实状态约 1.0 。更糟的是，策略会根据错误状态选择不同动作，误差不再只是线性相加。

因此模型内预测越长，Agent 越可能找到“只在错误模型里高奖励”的动作序列。这叫 Model Exploitation（模型利用）。

11.4.4 常见缓解方式

- 使用短 Model Rollout，减少误差累积。
- 使用 Model Ensemble（模型集成）估计不确定性。
- 对高不确定区域降低预测 Reward 或停止规划。
- 使用 MPC，每个真实步重新规划。
- 让真实数据持续覆盖策略即将访问的区域。

这些方法共同遵循一个原则：模型的价值取决于策略访问区域内的准确性，而不是测试集上的平均一步误差。

11.4.5 一个机械臂实操闭环

1. 收集随机策略和旧控制器轨迹
2. 训练模型预测下一关节状态和任务 Reward
3. 当前状态下采样多组未来力矩序列
4. 在模型中预测 H 步累计 Reward
5. 执行最优序列的第一个力矩
6. 将真实 Transition 加入数据集并重训模型

诊断时同时看真实一步预测误差、不同 Horizon 的 Rollout 误差、模型分歧，以及 Model Return 与真实 Evaluation Return 的差距。

11.5 Offline RL：为什么普通 Off-policy 算法不能直接套日志

11.5.1 Offline 不等于 Off-policy

- Off-policy：训练可用旧策略数据，但通常仍能继续交互补充 Replay Buffer。
- Offline RL：训练期间不能向环境请求任何新反馈， D 固定不变。

数据形式为：

$$D=\{(s,a,r,s',d)\}$$

仅仅“使用 Replay Buffer”不能说明算法适合 Offline RL。关键是策略选出数据外动作后，无法向真实环境验证并纠错。

11.5.2 标准 Q-learning 怎样形成外推错误闭环

假设某状态的数据只包含动作 a_1, a_2 ：

动作	是否有数据	Critic 估计
a_1	100 条	$Q=4.8$
a_2	80 条	$Q=5.1$
a_3	0 条	$Q=8.7$

a_3 的 8.7 没有真实依据，可能只是函数逼近外推。但普通 Policy Improvement 会选：

$$\arg \max_a Q(s, a) = a_3$$

随后 Bellman Target 还可能在下一状态继续使用数据外最大 Q：

无监督区域的 Q 偶然偏高

- > Policy 选择 OOD Action
- > Bootstrap Target 使用 OOD 高值
- > 高估在数据集内向前传播
- > 无真实交互可纠错

Offline RL 的几条主流路线，本质上都在切断这条链。

11.5.3 BCQ：先限制候选动作，再做最大化

BCQ 学习近似 Behavior Policy 的动作生成器。给定状态时：

1. 从生成器采样多个数据支持的候选动作。
2. 允许 Perturbation Network（扰动网络）做小幅调整。
3. 只在这些候选动作中选择最大 Q。

普通 Q-learning：在整个动作空间 max

BCQ：在 behavior-supported candidates 内 max

它降低 OOD 风险，但若数据本身质量差或覆盖窄，Policy 也难以超出 Behavior Policy 太多。

11.5.4 CQL：让没有数据支持的高 Q 付出额外代价

第一次阅读先记目标，不必先记公式：

Bellman Loss：让数据动作的 Q 符合真实 transition

CQL Penalty：让所有候选动作中的虚高 Q 付出代价

二者相加：数据动作正常拟合，数据外动作相对更保守

离散动作下，CQL 的直观目标可写成：

$$L_{CQL} = L_{Bellman} + \alpha_C [\log \sum_a \exp Q_w(s, a) - \mathbb{E}_{a \sim D(\cdot|s)} Q_w(s, a)]$$

α_C 是 Conservative Penalty Coefficient（保守惩罚系数）。第一项中的 Log-sum-exp 会对所有高 Q 动作施加压力；第二项通过负号相对保留数据动作的 Q。

若某 OOD 动作 Q 很高，它在 Log-sum-exp 的 Softmax 权重很大，却没有第二项的数据动作补偿，梯度下降会重点压低它。CQL 的核心不是“所有 Q 都变小”，而是让数据外动作相对数据动作更保守。

一个三动作例子：数据集在状态 s 只执行过 a_1 ，但 Critic 当前给出 $Q=(3, 4, 9)$ 。Log-sum-exp 几乎由 a_3 的 9 主导，因此第一项会强烈压低 a_3 ；第二项只补偿数据动作 a_1 。最终不是断言 a_3 一定差，而是在“没有证据”时拒绝相信它异常高的估值。

11.5.5 IQL：不生成数据外动作，也能偏向好数据

IQL 第一步只看数据动作的 $Q_w(s, a)$ ，通过 Expectile Regression（期望分位回归）学习 $V_v(s)$ 。令残差：

$$u = Q_w(s, a) - V_v(s)$$

损失为：

$$L_V(v) = \mathbb{E}_{(s,a) \sim D} [|\tau_e - 1[u < 0]| u^2]$$

当 $\tau_e > 0.5$ ：

- $Q > V$ 时权重是 τ_e ，低估高 Q 动作的代价较大；
- $Q < V$ 时权重是 $1 - \tau_e$ ，高估低 Q 动作的代价较小。

所以 V 会向数据动作 Q 分布的较高一侧移动，但没有对任意 OOD 动作取 $\max$ 。

两动作数值例子。 若同一状态下数据动作的 Q 分别为 2 和 8， $\tau_e = 0.8$ ，并假设 V 位于两者之间，则：

$$L(V) = 0.2(2-V)^2 + 0.8(8-V)^2$$

令导数为 0：

$$0.2(V-2) + 0.8(V-8) = 0$$

得到 $V = 6.8$ ，明显高于普通均值 5，但仍没有凭空查询第三个动作。

第二步用 V 构造 Q Target：

$$y = r + \gamma(1-d)V_v(s')$$

第三步用 Advantage-weighted BC（优势加权行为克隆）训练策略：

$$L_\pi(\theta) = -\mathbb{E}_{(s,a) \sim D} [\exp(\beta_I(Q_w(s,a) - V_v(s))) \log \pi_\theta(a|s)]$$

数据中高优势动作被更强地模仿，低优势动作权重更小。

把三步放回训练循环就不容易迷失：

1. 固定当前 Q ，用数据动作的 Q 拟合偏高侧的 V
2. 固定 V ，用 $r + \gamma V(s')$ 拟合 Q
3. 固定 Q/V ，用 $\exp(\beta * (Q - V))$ 加权模仿数据动作
4. 重复 1-3

这里 Q 和 V 看起来互相定义，但训练不是要求一次联立求出精确解，而是像普通 Actor-Critic 一样交替更新。关键边界仍是：每一步只使用数据集出现过的动作。

11.5.6 Decision Transformer：不做 Bellman 最大化，改做条件序列预测

DT 把一条轨迹排成：

$(RTG_1, s_1, a_1), (RTG_2, s_2, a_2), \dots$

训练时根据目标 RTG、历史状态和动作预测下一个动作。部署时：

1. 指定期望 RTG。
2. 输入当前历史，预测动作。
3. 环境返回 Reward 后，从剩余 RTG 中减去该 Reward。
4. 把新状态继续追加到上下文。

例如目标总回报为 10，第一步实际得到 3，则下一步条件 RTG 变为 7。DT 避免了 OOD 动作上的 Bellman max ，但不能凭目标 RTG 生成数据中完全不存在的动作；其表现仍受轨迹覆盖和条件可实现性约束。

11.5.7 四种 Offline 路线怎么选

方法	改动位置	核心保护	主要代价
BCQ	候选动作集合	只选数据支持动作	难以远离 Behavior
CQL	Q Loss	压低 OOD 高值	可能过度保守
IQL	Value 与 Policy 学习	全程使用数据动作	超参影响偏好强度
DT	问题表述	条件序列预测，无 Bellman Max	强依赖轨迹覆盖和条件设计

Offline 实操首先应做的是数据审计：状态动作覆盖、Behavior Policy 混合、Reward 缺失、Terminal 语义和 Trajectory Boundary。算法不能修复根本不存在的信息。

11.6 Imitation Learning：专家动作不是万能监督标签

11.6.1 Behavior Cloning 是什么

给定专家数据 (s, a_{expert}) ，BC 使用 Maximum Likelihood Estimation (MLE，最大似然估计)：

$$L_{BC}(\theta) = -\mathbb{E}_{(s, a_{expert}) \sim D} [\log \pi_{\theta}(a_{expert} | s)]$$

它没有 Bellman Target，也不需要 Reward。对离散动作是分类，对连续动作通常是概率回归。

11.6.2 为什么单步分类准确率高，闭环仍可能失败

BC 的训练状态来自专家分布。部署时一旦 Learner 犯错，可能进入专家数据中少见的状态；在这个状态再次预测不准，又进入更远的状态。

专家状态上小误差

- > 进入偏离轨迹的状态
- > 该状态没有训练覆盖
- > 更大动作误差
- > 状态进一步偏离

这叫 Covariate Shift（协变量偏移）和 Compounding Error（误差累积）。所以闭环任务不能只报告离线动作准确率，还要报告 Rollout Success Rate、碰撞率或轨迹偏差。

11.6.3 DAgger 怎样修复状态分布

DAgger 的流程是：

1. 用当前 Learner 与环境交互。
2. 对 Learner 实际访问的状态，请专家给正确动作。
3. 将新标注加入数据集。
4. 重新训练 Learner，重复上述过程。

关键变化是训练状态逐渐接近 Learner 的部署分布。代价是需要可重复调用专家，并且让早期 Learner 在线运行可能有安全风险。

11.6.4 IRL 与 BC 的区别

BC 直接学习“专家在这个状态做什么”；IRL 试图解释“专家为什么这样做”，先恢复一个 Reward Function，再求该 Reward 下的策略。

IRL 可能在新环境约束下重新规划，但 Reward 存在非唯一性：多个 Reward 都可能解释同一组专家行为，因此需要额外先验或正则化。

11.7 Preference Alignment：从偏好数据到 PPO-RLHF 与 DPO

11.7.1 先分清两条训练路线

PPO-RLHF：

SFT policy

- > 对同一 prompt 采样多个回答
- > 人类排序，训练 Reward Model
- > 当前 policy 在线生成回答
- > RM 给序列奖励，PPO 更新 policy
- > KL 约束 policy 不要远离 reference

DPO：

固定的 (prompt, preferred, rejected) 数据

- > 当前 policy 与 reference 计算回答 log probability
- > 直接最小化 pairwise preference loss

PPO-RLHF 有当前策略生成的新 Rollout；DPO 通常是固定偏好数据上的监督式优化。二者不能只因为都使用偏好数据就视为同一个训练过程。

11.7.2 Reward Model 怎样从偏好对学习

对 Prompt x 的偏好回答 y^+ 和被拒回答 y^- ，Bradley-Terry Model（Bradley-Terry 成对偏好模型）写成：

$$P(y^+ > y^- / x) = \sigma(r_\omega(x, y^+) - r_\omega(x, y^-))$$

RM Loss 为：

$$L_{RM}(\omega) = -\mathbb{E} [\log \sigma(r_\omega(x, y^+) - r_\omega(x, y^-))]$$

它只要求偏好回答的分数高于被拒回答，不要求分数具有绝对物理意义。

11.7.3 PPO-RLHF 的 Reward 与 KL 约束

语言模型的一次 Action 通常是生成一个 Token，但 RM 往往对整段回答给一个序列级分数。实践中 Critic 估计每个 Token 状态的未来价值，GAE（Generalized Advantage Estimation，广义优势估计）把序列 Reward 传播成逐 Token 的 Advantage。

常见总 Reward 直觉是：

$$r_{total}(x, y) = r_\omega(x, y) - \beta_{KL} \log \pi_\theta(y|x) / \pi_{ref}(y|x)$$

β_{KL} 是 KL Penalty Coefficient（KL 惩罚系数）。这项限制策略为追求 RM 分数而远离语言先验，降低 Reward Hacking 和语言退化风险。

这里有两个不同约束：

- PPO Clip 限制当前更新相对采样旧策略的变化。
- Reference KL 限制当前策略相对固定参考模型的长期漂移。

两者参照对象不同，不能互相替代。

11.7.4 DPO 从哪里推出来

先看最短逻辑链，后面的公式只是把这三句话写严谨：

人类偏好模型只关心两个回答的 Reward 差

KL 正则化最优策略把 Reward 写成 policy/reference 的 log-ratio

把 Reward 差代回偏好模型，就得到无需显式 Reward Model 的 DPO Loss

考虑 KL 正则化的单 Prompt 目标：

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} [r(x,y)] - \beta_D D_{KL}(\pi(\cdot|x) \parallel \pi_{ref}(\cdot|x))$$

为什么其最优策略具有下面的指数形式？先把固定 Prompt 下每个回答 y 的概率记为 π_y ，参考概率记为 q_y 。加上概率和为 1 的约束，Lagrangian（拉格朗日函数）为：

$$L(\pi, \lambda) = \sum_y \pi_y r_y - \beta_D \sum_y \pi_y \log \pi_y / q_y + \lambda (\sum_y \pi_y - 1)$$

λ 是 Lagrange Multiplier（拉格朗日乘子），只用于强制所有回答概率之和等于 1。

对每个 π_y 求偏导并令其为 0：

$$r_y - \beta_D (\log \pi_y / q_y + 1) + \lambda = 0$$

移项后可得：

$$\pi_y = q_y \exp(r_y / \beta_D) \times C(x)$$

$C(x)$ 收集了对同一 Prompt 所有回答都相同的常数。再用 $\sum_y \pi_y = 1$ 归一化， $C(x) = 1/Z(x)$ ，因此最优策略满足：

$$\pi^*(y|x) = 1 / Z(x) \pi_{ref}(y|x) \exp(r(x,y) / \beta_D)$$

整理得到 Reward 的等价表达：

$$r(x,y) = \beta_D \log \pi^*(y|x) / \pi_{ref}(y|x) + \beta_D \log Z(x)$$

同一个 Prompt 下做回答差值时， $\log Z(x)$ 抵消：

$$r(x,y^+) - r(x,y^-) = \beta_D [\log \pi^*(y^+|x) / \pi_{ref}(y^+|x) - \log \pi^*(y^-|x) / \pi_{ref}(y^-|x)]$$

把这个差值代入 Bradley-Terry 偏好概率，并用当前 π_θ 近似 π^* ，得到 DPO Loss：

$$L_{DPO}(\theta) = -\mathbb{E} [\log \sigma(\beta_D [\log \pi_\theta(y^+|x) / \pi_{ref}(y^+|x) - \log \pi_\theta(y^-|x) / \pi_{ref}(y^-|x)])]$$

DPO 不是“把 Chosen 的概率提高、Rejected 的概率降低”这么简单；它比较的是两者相对 Reference Policy 的 Log-ratio 差。

Loss 前为什么有负号？Bradley-Terry 给出“偏好回答胜出”的预测概率 p_θ ，训练希望最大化 $\log p_\theta$ ；深度学习框架使用梯度下降，所以最小化 $-\log p_\theta$ 。这与二分类交叉熵完全相同。

11.7.5 DPO 数值例子

设当前策略相对参考策略的序列 Log-ratio 为：

$$\log \pi_{\theta}(y^+ | x) / \pi_{ref}(y^+ | x) = 0.4$$

$$\log \pi_{\theta}(y^- | x) / \pi_{ref}(y^- | x) = -0.1$$

若 $\beta_D = 0.2$ ，Sigmoid 的输入为：

$$0.2 \times (0.4 - (-0.1)) = 0.1$$

偏好概率为 $\sigma(0.1) \approx 0.525$ 。训练会继续增大这两个 Log-ratio 的差值。若两者都同样相对 Reference 上升 0.3，差值不变，DPO 偏好概率也不变。

11.7.6 PPO-RLHF 与 DPO 的边界

维度	PPO-RLHF	DPO
训练信号	显式 RM 标量 Reward	成对偏好标签
数据	当前 Policy Rollout	通常固定偏好数据
Critic	需要	不需要
优化方式	RL Policy Optimization	监督式 Pairwise Loss
主要优势	可对当前策略的新样本持续优化	流程简单、稳定、资源需求较低
主要风险	RM 利用、训练复杂、采样昂贵	受固定偏好数据覆盖限制，不能在线查询 Reward

11.8 现实任务怎样判断属于哪一类

11.8.1 自动驾驶历史日志

若完全不能在线试错，但有 (s, a, r, s') 日志，这是 Offline RL；若只有人类驾驶动作而没有可信 Reward，先考虑 BC/DAGger。若另外学习车辆 Dynamics 做规划，则又加入 Model-based 成分。

11.8.2 推荐系统日志

日志只记录被旧系统展示的 Item 及反馈，未展示动作的 Reward 不可见。直接对全动作做 $\max Q$ 会有严重 OOD 问题，应关注 Support、Propensity（倾向概率）和保守策略改进，而不只是把在线 SAC 的 Replay Buffer 冻结。

11.8.3 聊天模型偏好对齐

只有固定 Chosen/Rejected 数据并训练单个 Policy，通常是 DPO 类离线偏好优化；如果当前 Policy 持续生成回答、RM 打分、Critic 估值并用 PPO 更新，则是 PPO-RLHF。

11.9 面试高频问题

11.9.1 Model-based RL 为什么样本效率高却可能不稳？

因为真实 Transition 可以训练模型，并在模型内产生大量便宜计算；但策略会利用模型误差，长 Horizon 还会累积误差，因此需短 Rollout、不确定性和真实数据校正。

11.9.2 Offline RL 与 Off-policy 有什么区别？

Off-policy 可以复用旧数据，但通常还能继续交互纠错；Offline RL 的数据集固定，新策略进入 OOD 区域后没有真实反馈可补救。

11.9.3 BCQ、CQL、IQL 的共同点和差别？

共同点是避免数据外高 Q 被策略利用。BCQ 限制候选动作，CQL 压低 OOD Q，IQL 不显式查询 OOD 动作并加权模仿数据中的好动作。

11.9.4 BC 为什么会有 Compounding Error？

训练状态来自专家，部署状态来自 Learner。一次小误差会把 Learner 带到训练分布外，后续错误继续累积；Dagger 用 Learner 访问状态上的专家标注缓解该问题。

11.9.5 DPO 是否属于 Actor-Critic？

不是。DPO 直接优化单个策略模型的偏好分类损失，没有单独 Critic，也不使用在线 Bellman Bootstrap。

11.9.6 PPO Clip 与 RLHF Reference KL 有什么区别？

PPO Clip 约束当前参数相对产生这批 Rollout 的旧策略；Reference KL 约束当前模型相对固定参考模型的长期漂移。

11.10 原始论文与参考资料

- Sutton, [Integrated Architectures for Learning, Planning, and Reacting Based on Approximating Dynamic Programming](#), Dyna.
- Fujimoto et al., [Off-Policy Deep Reinforcement Learning without Exploration](#), BCQ.
- Kumar et al., [Conservative Q-Learning for Offline Reinforcement Learning](#), CQL.
- Kostrikov et al., [Offline Reinforcement Learning with Implicit Q-Learning](#), IQL.
- Chen et al., [Decision Transformer: Reinforcement Learning via Sequence Modeling](#), DT.
- Ross et al., [A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning](#), Dagger.
- Ouyang et al., [Training Language Models to Follow Instructions with Human Feedback](#), PPO-RLHF pipeline.
- Rafailov et al., [Direct Preference Optimization: Your Language Model is Secretly a Reward Model](#), DPO.

11.11 本章练习

- 一个 Dynamics Model 的一步状态误差只有 0.05，为什么不能据此断言 100 步 Planning 可靠？列出两种缓解方式。
- 用“数据支持、Q 外推、是否继续交互”解释普通 SAC 为什么不能直接当 Offline SAC 使用。
- 对数据动作 Q 值 2 和 8，分别计算 IQL Expectile Level 为 0.5 和 0.8 时、且 V 位于两者之间的最优 V 。
- 解释 BC 的测试集动作准确率与闭环 Success Rate 为什么可能相反。
- 从 KL 正则化最优策略写到 DPO 的 Log-ratio 差，并说明 Reference Policy 为什么不能删掉。

▼ 参考答案（点击展开）

- 一步误差会改变后续预测状态，策略又会根据错误状态选择不同动作，误差可能非线性累积。可使用短 Model Rollout、MPC 重规划、模型集成不确定性或持续真实数据校正。
- 普通 SAC 的 Actor 会在连续动作空间寻找高 Q 动作；固定日志未覆盖处的 Q 可能只是函数外推，Soft Bellman Target 还会传播该高估。在线 SAC 尚可通过新交互纠错，Offline 场景不能，因此需动作约束、保守 Q 或 In-sample 学习。
- $\tau_e=0.5$ 时是对称平方损失， $V=5$ ； $\tau_e=0.8$ 时 $0.2(V-2)+0.8(V-8)=0$ ， $V=6.8$ 。
- 离线测试状态仍来自专家分布；部署时 Learner 的小错误改变后续状态分布，未覆盖状态上的错误会累积。因此单步准确率高并不保证长时闭环成功。
- 对固定 Prompt 的每个回答概率求导，可得 $r_y - \beta_D(\log(\pi_y/q_y)+1)+\lambda=0$ ，所以 $\pi^* \propto \pi_{ref} \exp(r/\beta_D)$ 。因此同 Prompt 的 Reward 差等于 β_D 乘两回答相对 Reference 的 Log-ratio 差。Reference 给出“相对原模型改变了多少”的基准；删掉后便失去原推导中的 KL 锚点。